

Họ và tên : Phạm Minh Đạt

MSSV : 20210169

Modulo phụ trách : Payment (Thanh toán PayPal & VietQR)

I. TỔNG QUAN

Báo cáo này tập trung đánh giá thiết kế hiện tại của module **Payment** (Thanh toán) và **Home** (Trang chủ) trong dự án AIMS. Các đánh giá dựa trên 5 nguyên lý SOLID, đặc biệt tập trung vào mức độ liên kết (Cohesion) để đánh giá SRP và khả năng mở rộng (Extensibility) để đánh giá OCP.

II. PHÂN TÍCH CHI TIẾT CÁC VI PHẠM

1. Module Payment

Module Payment đóng vai trò quan trọng trong việc xử lý giao dịch của khách hàng. Tuy nhiên, thiết kế hiện tại của lớp giao diện chính (PaymentForm) đang tồn tại nhiều vấn đề vi phạm các nguyên lý thiết kế cơ bản, dẫn đến việc khó bảo trì và mở rộng.

a. Vi phạm Nguyên lý Đơn nhiệm (Single Responsibility Principle - SRP)

- **Tình trạng:** Lớp PaymentForm đang có mức độ liên kết (Cohesion) thấp. Nó không chỉ chịu trách nhiệm hiển thị giao diện người dùng (UI Layer) mà còn bao hàm cả logic điều khiển nghiệp vụ (Business Logic/Controller Layer).
- **Chứng minh qua mã nguồn:** Trong phương thức createPayPalContent, logic xử lý sự kiện click chuột (setOnAction) chứa các đoạn mã xử lý nghiệp vụ như: khởi tạo hệ thống con, gọi API lấy URL, và tương tác với hệ điều hành để mở trình duyệt.

// File: PaymentForm.java (Old Version)

```
payBtn.setOnAction(e -> {
```

```
    try {
```

```
        // [VI PHẠM SRP]: View tự mình xử lý logic nghiệp vụ gọi API
```

```
        IQRCodePayment paypalSystem = new PayPalSubsystem();
```

```
        String payUrl = paypalSystem.generatePayUrl(this.amount, "Content");
```

```
        // [VI PHẠM SRP]: View chứa logic hệ thống (System Logic)
```

```

        java.awt.Desktop.getDesktop().browse(new java.net.URI(payUrl));

        // ... logic hiển thị Alert ...

    } catch (Exception ex) {
        ex.printStackTrace();
    }
});

```

Hậu quả: Khi logic thanh toán thay đổi (ví dụ: thay đổi cách xử lý kết quả trả về từ API), ta bắt buộc phải chỉnh sửa file giao diện PaymentForm. Điều này làm tăng rủi ro gây lỗi hiển thị khi sửa logic và ngược lại.

b. Vi phạm Nguyên lý Đóng/Mở (Open/Closed Principle - OCP)

- **Tình trạng:** Hệ thống không "đóng" với việc sửa đổi. Các phương thức thanh toán được gán cứng (Hard-coded) ngay trong code khởi tạo giao diện.
- **Kịch bản thay đổi:** Giả sử trong tương lai, hệ thống cần tích hợp thêm cổng thanh toán ZaloPay.

// File: PaymentForm.java (Old Version)

```

private void initializeUI() {
    TabPane paymentMethods = new TabPane();

    // [VI PHẠM OCP]: Các phương thức thanh toán bị khởi tạo cứng
    Tab qrTab = new Tab("Thanh toán qua VietQR");
    qrTab.setContent(createVietQRContent());

    Tab creditTab = new Tab("Thẻ tín dụng (PayPal)");
    creditTab.setContent(createPayPalContent());

```

```
// Nếu muốn thêm ZaloPay, bắt buộc phải SỬA CODE ở đây để thêm dòng:
// paymentMethods.getTabs().add(createZaloPayContent());

paymentMethods.getTabs().addAll(qrTab, creditTab);
// ...
}
```

Hậu quả: Mỗi khi có yêu cầu nghiệp vụ mới về phương thức thanh toán, lập trình viên buộc phải can thiệp vào mã nguồn cũ của PaymentForm, vi phạm nguyên tắc OCP và tăng khả năng phát sinh lỗi ở các chức năng đang chạy ổn định.

c. Vi phạm Nguyên lý Đảo ngược sự phụ thuộc (Dependency Inversion Principle - DIP)

- Tình trạng: Module cấp cao (PaymentForm - Giao diện) đang phụ thuộc trực tiếp vào Module cấp thấp (VietQRSubsystem, PayPalSubsystem - Triển khai chi tiết).
- Chứng minh qua mã nguồn: Việc sử dụng từ khóa new để khởi tạo đối tượng trực tiếp trong phương thức.

// File: PaymentForm.java (Old Version)

```
private VBox createVietQRContent() {
    try {
        // [VI PHẠM DIP]: Phụ thuộc chặt chẽ (Tight Coupling) vào class cụ thể
        // View biết quá nhiều về cách Subsystem được khởi tạo
        IQRCodePayment paymentSubsystem = new VietQRSubsystem();

        // ...
    } catch (Exception e) {
        // ...
    }

    return box;
}
```

```
}
```

Hậu quả:

1. **Tight Coupling:** PaymentForm bị dính chặt với VietQRSubsystem.
2. **Khó kiểm thử (Untestable):** Không thể viết Unit Test cho giao diện PaymentForm một cách độc lập vì không thể thay thế VietQRSubsystem bằng một Mock Object (đối tượng giả lập) để kiểm tra hành vi giao diện.

2.Module Subsystem

Các module con (Subsystem) đóng vai trò giao tiếp với hệ thống bên thứ ba (Third-party services). Tuy nhiên, việc thiết kế các interface và cách tổ chức dữ liệu cấu hình đang gặp một số vấn đề nghiêm trọng.

a. Vi phạm Nguyên lý Phân tách Interface (Interface Segregation Principle - ISP)

- **Tình trạng:** Hệ thống định nghĩa một Interface chung IQRCodePayment quá lớn ("Fat Interface"), bao gồm cả hai hành vi: generatePayUrl (Thanh toán) và refund (Hoàn tiền).
- **Chứng minh qua mã nguồn:** Lớp VietQRSubsystem (sử dụng tài khoản ngân hàng cá nhân) không hỗ trợ API hoàn tiền tự động. Tuy nhiên, do bị buộc phải implements IQRCodePayment, lớp này phải chứa một phương thức "giả" (Dummy Implementation).

// File: VietQRSubsystem.java (Old Version)

@Override

```
public String refund(int amount, String content) throws PaymentException {
```

```
    // [VI PHẠM ISP]: Phương thức vô nghĩa, chỉ để thỏa mãn Interface
```

```
    // VietQR cá nhân không thể refund tự động.
```

```
    return "Yêu cầu hoàn tiền thành công (VietQR - Manual Process)";
```

```
}
```

Hậu quả:

1. **Gây hiểu nhầm:** Client gọi hàm refund() của VietQR sẽ nhận được thông báo thành công dù thực tế tiền chưa được hoàn trả, dẫn đến sai lệch dữ liệu.

2. Dư thừa: Lớp con phải implement những phương thức mà nó không sử dụng.

b. Vi phạm Nguyên lý Đơn nhiệm (SRP) trong Controller

- Tình trạng: Lớp VietQRSubsystemController đang trộn lẫn giữa Logic nghiệp vụ (cách xây dựng URL string) và Dữ liệu cấu hình (thông tin tài khoản).

// File: VietQRSubsystemController.java (Old Version)

```
public class VietQRSubsystemController {  
  
    // [VI PHẠM SRP]: Chứa dữ liệu cấu hình cứng  
    private static final String BANK_ID = "ICB";  
    private static final String ACCOUNT_NO = "109875430178";  
    private static final String ACCOUNT_NAME = "PHAM MINH DAT";  
  
    public String generateQRUrl(...) {  
        // .....  
    }  
}
```

c. Vi phạm Nguyên lý Đảo ngược sự phụ thuộc (DIP) trong PayPal

- Tình trạng: Lớp PayPalSubsystem đóng vai trò là Facade nhưng lại tự mình khởi tạo đối tượng Controller cụ thể.
- Chứng minh qua mã nguồn:

// File: PayPalSubsystem.java (Old Version)

```
public class PayPalSubsystem implements IQRCodePayment {  
  
    private PayPalSubsystemController ctrl;  
  
    public PayPalSubsystem() {
```

```

// [VI PHẠM DIP]: Khởi tạo trực tiếp bằng từ khóa 'new'
this.ctrl = new PayPalSubsystemController();
}
// ...
}

```

Hậu quả: Tạo ra sự phụ thuộc cứng (Tight Coupling). Không thể thay thế PayPalSubsystemController bằng một phiên bản khác (hoặc Mock object) khi cần kiểm thử hoặc thay đổi hành vi.

d. Vấn đề Clean Code & Vi phạm OCP (Magic Numbers & Hard-coded Strings)

- Tình trạng: Logic tính toán tỷ giá và các đường dẫn URL được viết "cứng" (Hard-coded) ngay trong logic của Controller.

// File: PayPalSubsystemController.java (Old Version)

```

public String createOrder(int amount) {
    // [MAGIC NUMBER]: Số 24000.0 xuất hiện không rõ ràng
    double amountUSD = amount / 24000.0;

    // [HARD-CODED STRING]: URL localhost bị gán cứng
    applicationContext.addProperty("return_url", "http://localhost:8080/paypal-success");
    // ...
}

```

Hậu quả:

1. Khó bảo trì: Khi tỷ giá thay đổi, phải tìm và sửa trực tiếp trong code logic.
2. Rủi ro triển khai: Các URL localhost sẽ không hoạt động khi deploy lên server thật, đòi hỏi phải sửa lại code nguồn (Vi phạm OCP: không đóng gói với việc sửa đổi).

3. Module Home

Lớp HomeForm là màn hình chính của ứng dụng, chịu trách nhiệm hiển thị danh sách sản phẩm và điều hướng người dùng. Tuy nhiên, việc ôm đồm quá nhiều trách nhiệm hiển thị chi tiết và logic khởi tạo đã khiến lớp này vi phạm các nguyên lý SRP, OCP và DIP.

a. Vi phạm Nguyên lý Đơn nhiệm (Single Responsibility Principle - SRP)

- **Tình trạng:** HomeForm có độ liên kết (Cohesion) thấp. Nó vừa quản lý bố cục tổng thể của trang (Header, ScrollPane), vừa quản lý chi tiết cách hiển thị của từng thẻ sản phẩm con (Product Card). Ngoài ra, nó còn chứa logic điều hướng phức tạp.
- **Chứng minh qua mã nguồn:** Phương thức createProductCard nằm trực tiếp trong HomeForm. Điều này biến HomeForm thành một lớp "God Class" quản lý từ cái lớn nhất đến cái nhỏ nhất.

// File: HomeForm.java (Old Version)

```
private VBox createProductCard(Media media) {  
  
    // [VI PHẠM SRP]: HomeForm phải lo chi tiết từng Label, Button của thẻ sản phẩm.  
  
    // Nếu muốn đổi màu nút "Add to Cart", ta phải sửa HomeForm (vốn là nơi quản lý trang chủ).  
  
    VBox card = new VBox(10);  
  
    // ... code tạo Image, Label, Button ...  
  
    return card;  
}
```

Hậu quả: Class trở nên cồng kềnh, khó đọc. Bất kỳ thay đổi nhỏ nào về giao diện thẻ sản phẩm cũng yêu cầu phải biên dịch lại toàn bộ lớp Trang chủ.

b. Vi phạm Nguyên lý Đóng/Mở (Open/Closed Principle - OCP)

- **Tình trạng:** Phương thức hiển thị sản phẩm không đóng với việc sửa đổi.

- Kịch bản thay đổi: Hệ thống có thêm loại sản phẩm mới là E-Book (Sách điện tử). Yêu cầu hiển thị nút "Read Now" thay vì "Add to Cart", và hiển thị "Author" thay vì "Category".
- Chứng minh qua mã nguồn: Hiện tại, `createProductCard` xử lý mọi Media như nhau. Để hỗ trợ yêu cầu trên, ta buộc phải sửa code để thêm if-else:

// File: HomeForm.java (Old Version)

```
private VBox createProductCard(Media media) {
    // ...
    // [VI PHẠM OCP]: Phải sửa code cũ để thêm logic mới
    if (media instanceof Book) {
        // Hiển thị kiểu Book
    } else if (media instanceof DVD) {
        // Hiển thị kiểu DVD
    }
    // ...
}
```

Hậu quả: Vi phạm OCP khiến mã nguồn trở nên mong manh, dễ vỡ khi thêm tính năng mới. Việc sửa đổi HomeForm có thể vô tình làm hỏng giao diện của các sản phẩm cũ (CD, DVD).

c. Vi phạm Nguyên lý Đảo ngược sự phụ thuộc (Dependency Inversion Principle - DIP)

- Tình trạng: View (HomeForm) phụ thuộc trực tiếp vào Controller cụ thể (HomeController) thay vì một Abstraction.

// File: HomeForm.java (Old Version)

```
public class HomeForm extends BaseForm {
    private HomeController homeController;
```



```

public HomeForm() {
    // [VI PHẠM DIP]: Khởi tạo trực tiếp Controller bằng từ khóa 'new'
    // View bị dính chặt với logic lấy dữ liệu từ DB thật.
    this.homeController = new HomeController();
}
// ...
}

```

III. Giải Pháp Refactoring

1. Tách Controller

Mục tiêu:

- Loại bỏ logic nghiệp vụ ra khỏi lớp giao diện PaymentForm.
- Giảm sự phụ thuộc chặt chẽ giữa PaymentForm và các Subsystem.

Giải pháp: Tôi đã tạo ra lớp điều khiển trung gian PaymentScreenHandler.

- PaymentForm (View): Chỉ chịu trách nhiệm hiển thị và chuyển tiếp sự kiện người dùng đến Handler.
- PaymentScreenHandler (Controller): Chịu trách nhiệm khởi tạo Subsystem, gọi API và xử lý dữ liệu trả về.

Kết quả:

- Đạt chuẩn SRP: View và Logic tách biệt.
- Đạt chuẩn DIP: View không còn phụ thuộc trực tiếp vào Subsystem.

2. Tách Config

Mục tiêu:

- Loại bỏ các con số ma thuật (Magic Numbers) và chuỗi cứng (Hard-coded Strings).
- Tách dữ liệu cấu hình ra khỏi logic xử lý.

Giải pháp: Tạo các lớp cấu hình riêng biệt (Configs, PayPalConfig, VietQRConfig) để chứa các hằng số.

Kết quả:

- Đạt chuẩn SRP: Logic xử lý và Dữ liệu cấu hình được tách riêng.
- Cải thiện OCP: Khi thay đổi tỷ giá hoặc môi trường deploy, chỉ cần sửa file Config, không cần chạm vào code Logic.

3. Tách Interface

Mục tiêu:

- Khắc phục tình trạng "Fat Interface" của IQRCodePayment.
- Tránh việc VietQRSubsystem phải implement phương thức refund giả.

Giải pháp đề xuất: Chia nhỏ IQRCodePayment thành các Interfaces chuyên biệt hơn theo chức năng.

Thiết kế đề xuất:

Interface IPayable: Chỉ chứa phương thức thanh toán.

Interface IRefundable: Chỉ chứa phương thức hoàn tiền.

Triển khai (Implementation):

- VietQRSubsystem: Chỉ implements IPayable.
 - -> Không còn phải viết hàm refund thừa thãi.
- PayPalSubsystem: Implements cả IPayable và IRefundable.

Kết quả:

-Đạt chuẩn ISP: Các lớp con chỉ phải implement những phương thức mà nó thực sự cần dùng. Hệ thống trở nên linh hoạt và đúng ngữ nghĩa hơn.

IV. KẾT LUẬN

Qua quá trình đánh giá và tái cấu trúc (Refactoring), module Payment và Home của dự án AIMS đã có những cải thiện rõ rệt:

1. Tính liên kết (Cohesion) tăng cao: Mỗi lớp (Form, Handler, Controller, Config) đều tập trung vào một nhiệm vụ duy nhất.
2. Sự phụ thuộc (Coupling) giảm xuống: Giao diện không còn bị dính chặt với logic nghiệp vụ hay các thư viện bên thứ ba.
3. Khả năng bảo trì và mở rộng: Dễ dàng thêm phương thức thanh toán mới, thay đổi cấu hình hoặc viết Unit Test cho từng thành phần riêng biệt.

